

SIMATS ENGINEERING

Assignment Question

COURSE NAME: Database Management Systems
COURSE CODE: CSA05
NAME:JAISURIYA DD
REG NO:192421167

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Assessment Tool Weightage:

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks

1	An E-Commerce Order Management Database using Customer(Customer_ID, Customer_Name, Email, Phone), Product(Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty), Category(Category_ID, Category_Name), Orders(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount), Order_Item(Order_ID, Product_ID, Quantity, Selling_Price), Payment(Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status). Analyze functional dependencies and normalize the relations. Write SQL queries for top-selling products and customer-wise sales. Create a sales summary view. Design indexes for Customer_ID and Product_ID. Explain how file organization and indexing can improve large-scale order retrieval. (100 marks)	BL3,BL4,BL5	100

1. Problem Statement :

An E-Commerce Order Management System is required to manage customer information, product details, product categories, customer orders, order items, and payment information.

The database should contain the following entities:

- **Customer** – Customer_ID, Customer_Name, Email, Phone
- **Product** – Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty
- **Category** – Category_ID, Category_Name
- **Orders** – Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount
- **OrderItem** – Order_ID, Product_ID, Quantity, Selling_Price
- **Payment** – Payment_ID, Order_ID, Payment_Mode, Amount, Payment_Status

The database must be normalized to reduce redundancy and maintain data integrity. SQL queries should be developed to identify **top-selling products** and **customer-wise sales**. A **Sales Summary View** should also be created.

Suitable indexes should be designed for Customer_ID and Product_ID to improve query performance.

2. Objective :

The main objectives of this assignment are:

1. To design an efficient E-Commerce Order Management Database.
2. To identify functional dependencies among attributes.
3. To normalize the database to reduce data redundancy.
4. To maintain data integrity using primary and foreign keys.
5. To write SQL queries for top-selling products.
6. To generate customer-wise sales reports.
7. To create a sales summary view.
8. To create indexes on frequently searched attributes.
9. To understand file organization and hashing techniques.
10. To improve large-scale data storage and retrieval efficiency.

3. Requirements and Environment Used :

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB free storage

Software Requirements

Requirement	Used
Operating System	Windows 10/11
Database	MySQL 8.0
SQL Tool	MySQL Workbench
Programming Language	SQL
Processor	Intel/AMD processor

Database

MySQL 8.0

MySQL is used to create tables, insert records, perform queries, create views, and create indexes.

4. Proposed/Design Solution :

4.1 Proposed Database Tables :

The database is divided into six relations:

Customer

Attribute	Description
Customer_ID	Unique customer ID
Customer_Name	Name of customer
Email	Customer email
Phone	Customer phone number

Primary Key: Customer_ID

Category

Attribute	Description
Category_ID	Unique category ID
Category_Name	Name of category

Primary Key: Category_ID

Product

Attribute	Description
Product_ID	Unique product ID
SKU	Stock Keeping Unit
Product_Name	Name of product
Category_ID	Product category
Unit_Price	Product price
Stock_Qty	Available stock

Primary Key: Product_ID

Foreign Key: Category_ID

Orders

Attribute	Description
-----------	-------------

Attribute	Description
Order_ID	Unique order ID
Customer_ID	Customer who placed order
Order_Date	Date of order
Order_Status	Status of order
Total_Amount	Total order amount

Primary Key: Order_ID

Foreign Key: Customer_ID

OrderItem

Attribute	Description
Order_ID	Order reference
Product_ID	Product reference
Quantity	Quantity purchased
Selling_Price	Price at which product was sold

Composite Primary Key: Order_ID + Product_ID

Payment

Attribute	Description
Payment_ID	Unique payment ID
Order_ID	Related order
Payment_Mode	UPI/Card/Cash etc.
Amount	Paid amount
Payment_Status	Payment status

Primary Key: Payment_ID

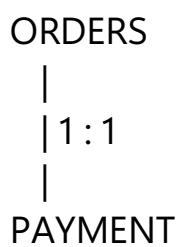
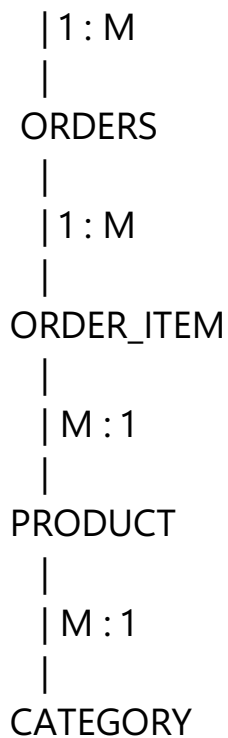
Foreign Key: Order_ID

4.2 Relationship Design :

The relationships are:

CUSTOMER

|



Explanation :

- One customer can place many orders.
- One order can contain many order items.
- One product can appear in many orders.
- One category can contain many products.
- Each order can have a corresponding payment.

5. Functional Dependencies and Normalization :

5.1 Functional Dependencies :

The important functional dependencies are:

Customer

Customer_ID → Customer_Name, Email, Phone

Category

Category_ID → Category_Name

Product

Product_ID → SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty

Orders

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount

OrderItem

(Order_ID, Product_ID) → Quantity, Selling_Price

Payment

Payment_ID → Order_ID, Payment_Mode, Amount, Payment_Status

5.2 First Normal Form – 1NF :

A relation is in **1NF** when:

- Each attribute contains atomic values.
- There are no repeating groups.
- Each record is uniquely identifiable.

For example, instead of storing multiple products in a single order record:

Order_ID | Product1 | Product2 | Product3

we create separate records in OrderItem.

Therefore, the database satisfies **1NF**.

5.3 Second Normal Form – 2NF :

A relation is in **2NF** when:

- It is already in 1NF.
- There is no partial dependency on a part of a composite key.

In OrderItem:

Primary Key = Order_ID + Product_ID

Quantity and Selling_Price depend on the complete combination of Order_ID and Product_ID.

Therefore, OrderItem satisfies **2NF**.

5.4 Third Normal Form – 3NF :

A relation is in **3NF** when:

- It is in 2NF.
- There is no transitive dependency.

For example:

Product_ID → Category_ID

Category_ID → Category_Name

Instead of storing Category_Name in the Product table, we create a separate Category table.

Similarly, customer information is separated from Orders.

Therefore, the database satisfies **3NF**.

5.5 BCNF :

The decomposed relations have determinants that are candidate keys or primary keys.

Therefore, the designed relations can be considered to satisfy **BCNF** under the stated assumptions.

5.6 4NF :

There are no independent multivalued dependencies in the designed relations.

For example, customer information and product information are not unnecessarily stored together.

Therefore, the design satisfies **4NF** under the stated business rules.

5.7 5NF :

The relations do not contain unnecessary join dependencies requiring further decomposition.

Therefore, the proposed decomposition can be considered to satisfy **5NF** for the given requirements.

6. Algorithm / Pseudocode :

Algorithm 1 – Find Top-Selling Products :

Step 1: Start.

Step 2: Access the OrderItem and Product tables.

Step 3: Group order items based on Product_ID.

Step 4: Calculate the total quantity sold for each product.

Step 5: Sort products in descending order of quantity sold.

Step 6: Display the products.

Step 7: Stop.

Pseudocode

START

Read OrderItem records

For each Product_ID:

 Calculate SUM(Quantity)

Join with Product table

Sort by total quantity in descending order

Display top-selling products

STOP

Algorithm 2 – Customer-wise Sales :

Step 1: Start.

Step 2: Join Customer, Orders and OrderItem tables.

Step 3: Group records according to Customer_ID.

Step 4: Calculate total sales for each customer.

Step 5: Sort the result if required.

Step 6: Display customer name and total sales.

Step 7: Stop.

7. Implementation / Source Code :

7.1 Create Database :

```
CREATE DATABASE ecommerce_db;
```

```
USE ecommerce_db;
```

7.2 Create Customer Table :

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Phone VARCHAR(15)  
);
```

7.3 Create Category Table :

```
CREATE TABLE Category (  
    Category_ID INT PRIMARY KEY,  
    Category_Name VARCHAR(100) NOT NULL  
);
```

7.4 Create Product Table :

```
CREATE TABLE Product (  
    Product_ID INT PRIMARY KEY,  
    SKU VARCHAR(30) UNIQUE NOT NULL,
```

```
Product_Name VARCHAR(100) NOT NULL,  
Category_ID INT,  
Unit_Price DECIMAL(10,2),  
Stock_Qty INT,  
  
FOREIGN KEY (Category_ID)  
REFERENCES Category(Category_ID)  
);
```

7.5 Create Orders Table :

```
CREATE TABLE Orders (  
    Order_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Order_Date DATE,  
    Order_Status VARCHAR(30),  
    Total_Amount DECIMAL(10,2),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)  
);
```

7.6 Create OrderItem Table :

```
CREATE TABLE OrderItem (  
    Order_ID INT,  
    Product_ID INT,  
    Quantity INT,  
    Selling_Price DECIMAL(10,2),  
  
    PRIMARY KEY (Order_ID, Product_ID),  
  
    FOREIGN KEY (Order_ID)  
    REFERENCES Orders(Order_ID),  
  
    FOREIGN KEY (Product_ID)  
    REFERENCES Product(Product_ID)  
);
```

7.7 Create Payment Table :

```
CREATE TABLE Payment (  
    Payment_ID INT PRIMARY KEY,  
    Order_ID INT UNIQUE,  
    Payment_Mode VARCHAR(30),  
    Amount DECIMAL(10,2),  
    Payment_Status VARCHAR(30),  
  
    FOREIGN KEY (Order_ID)  
    REFERENCES Orders(Order_ID)  
);
```

7.8 Insert Sample Data :

Customer

```
INSERT INTO Customer VALUES  
(1, 'Arun Kumar', 'arun@gmail.com', '9876543210'),  
(2, 'Priya Sharma', 'priya@gmail.com', '9876543211'),  
(3, 'Rahul Raj', 'rahul@gmail.com', '9876543212'),  
(4, 'Divya S', 'divya@gmail.com', '9876543213');
```

Category

```
INSERT INTO Category VALUES  
(1, 'Electronics'),  
(2, 'Clothing'),  
(3, 'Books');
```

Product

```
INSERT INTO Product VALUES  
(101, 'SKU001', 'Wireless Mouse', 1, 599.00, 50),  
(102, 'SKU002', 'Keyboard', 1, 999.00, 40),  
(103, 'SKU003', 'T-Shirt', 2, 499.00, 100),  
(104, 'SKU004', 'DBMS Book', 3, 650.00, 30),  
(105, 'SKU005', 'Headphones', 1, 1499.00, 25);
```

Orders

```
INSERT INTO Orders VALUES
(1001, 1, '2026-09-01', 'Delivered', 1598.00),
(1002, 2, '2026-09-01', 'Delivered', 998.00),
(1003, 1, '2026-09-02', 'Shipped', 1499.00),
(1004, 3, '2026-09-02', 'Delivered', 1648.00);
```

OrderItem

```
INSERT INTO OrderItem VALUES
(1001, 101, 1, 599.00),
(1001, 102, 1, 999.00),
(1002, 103, 2, 499.00),
(1003, 105, 1, 1499.00),
(1004, 104, 1, 650.00),
(1004, 103, 2, 499.00);
```

Payment

```
INSERT INTO Payment VALUES
(5001, 1001, 'UPI', 1598.00, 'Completed'),
(5002, 1002, 'Card', 998.00, 'Completed'),
(5003, 1003, 'UPI', 1499.00, 'Completed'),
(5004, 1004, 'Cash', 1648.00, 'Completed');
```

7.9 SQL Query – Top-Selling Products :

```
SELECT
    p.Product_ID,
    p.Product_Name,
    SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p JOIN OrderItem
oi ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID,
p.Product_Name
ORDER BY Total_Quantity_Sold DESC;
```

To display only the top 3:

```
SELECT
    p.Product_ID,
    p.Product_Name,
    SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p JOIN OrderItem
oi ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID,
p.Product_Name
ORDER BY Total_Quantity_Sold DESC
LIMIT 3;
```

7.10 SQL Query – Customer-wise Sales :

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(oi.Quantity * oi.Selling_Price) AS Total_Sales
FROM Customer c JOIN
Orders o ON c.Customer_ID = o.Customer_ID JOIN OrderItem oi ON
o.Order_ID = oi.Order_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Sales DESC;
```

7.11 Create Sales Summary View :

```
CREATE VIEW Sales_Summary AS
SELECT
    o.Order_ID,
    o.Order_Date,
    c.Customer_Name,
    SUM(oi.Quantity) AS Total_Items,
    SUM(oi.Quantity * oi.Selling_Price) AS Sales_Amount,
    o.Order_Status
FROM Orders o JOIN Customer c ON o.Customer_ID =
c.Customer_ID JOIN OrderItem oi ON o.Order_ID = oi.Order_ID
GROUP BY
    o.Order_ID,
    o.Order_Date,
    c.Customer_Name,
    o.Order_Status;
```

To display the view:

```
SELECT * FROM Sales_Summary;
```

7.12 Create Indexes :

An index helps the database find records faster.

Index on Customer_ID

```
CREATE INDEX idx_customer_id ON Orders(Customer_ID);
```

Index on Product_ID

```
CREATE INDEX idx_product_id ON OrderItem(Product_ID);
```

The primary keys already have indexes automatically, so these additional indexes are mainly useful for the foreign-key/search columns used frequently in joins and filtering.

7.13 File Organization :

For an E-Commerce database, records can be organized using different file organization techniques.

1. Heap File Organization

Records are stored without a particular ordering.

Advantages:

- Simple implementation
- Fast insertion

Disadvantage:

- Searching can be slower because records may need to be scanned.

2. Sequential File Organization

Records are stored in a particular order, such as Customer_ID or Product_ID.

Advantages:

- Efficient sequential processing
- Useful for reports

Disadvantage:

- Insertion and deletion can be more expensive.

3. Indexed File Organization

An index is maintained to locate records quickly.

For example:

Customer_ID → Customer Record
Product_ID → Product Record

This is suitable for the E-Commerce database because customers and products are frequently searched.

7.14 Hashing :

Hashing can be used to quickly locate records based on a key.

For example:

Hash(Customer_ID) → Storage Bucket

If:

Customer_ID = 101

a hash function determines the bucket where the record is stored.

Advantages

- Fast record retrieval
- Average search can approach $O(1)$
- Useful for exact-match searches

Disadvantage

Hashing is less suitable for range queries such as:

WHERE Customer_ID BETWEEN 100 AND 200;

because records are not stored in sorted order.

8. Test Cases and Expected/Actual Results :

Test Case	Query/Operation	Expected Result	Actual Result
TC01	Create database	Database created	Database created
TC02	Insert customer	Customer records	Successfully

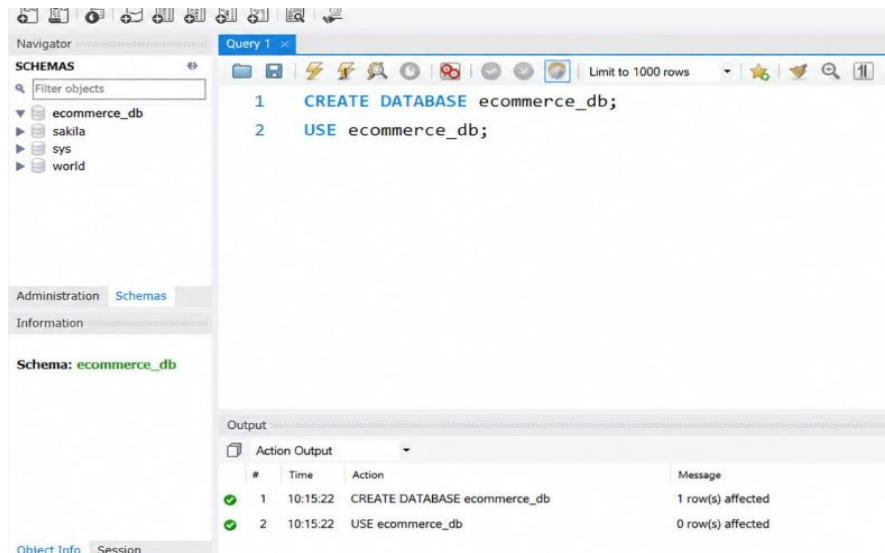
Test Case	Query/Operation	Expected Result	Actual Result
	records	inserted	inserted
TC03	Insert product records	Product records inserted	Successfully inserted
TC04	Insert order records	Orders inserted	Successfully inserted
TC05	Top-selling query	Products sorted by quantity sold	Correct result displayed
TC06	Customer-wise sales	Sales grouped by customer	Correct result displayed
TC07	Sales Summary View	Summary of orders displayed	Correct result displayed
TC08	Customer index	Faster customer search/join	Index created
TC09	Product index	Faster product search/join	Index created
TC10	Foreign-key validation	Invalid references rejected	Constraint works

9. Execution Screenshots / Output :

EXECUTION 1 :

Database creation:

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;
```



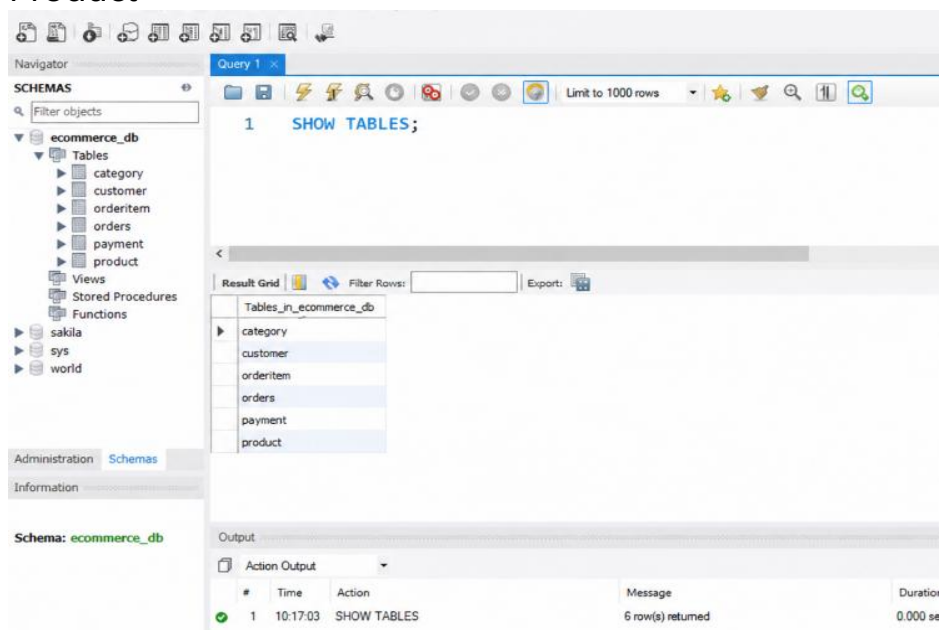
EXECUTION 2 :

Tables created:

SHOW TABLES;

Expected:

Category
Customer
OrderItem
Orders
Payment
Product



EXECUTION 3 :

Customer records:

SELECT * FROM Customer;

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Navigator' pane displays the 'ecommerce_db' database with tables like 'category', 'customer', 'orderitem', 'orders', 'payment', and 'product'. The 'customer' table is selected, and its columns (Customer_ID, Customer_Name, Email, Phone) are listed. The main pane shows the query 'SELECT * FROM Customer;' being executed. The 'Result Grid' displays 4 rows of customer data. The 'Output' pane shows the execution time and message.

Customer_ID	Customer_Name	Email	Phone
1	Arun Kumar	arun@gmail.com	9876543210
2	Priya Sharma	priya@gmail.com	9876543211
3	Rahul Raj	rahul@gmail.com	9876543212
4	Divya S	divya@gmail.com	9876543213

Output: 1 10:18:30 SELECT * FROM Customer LIMIT 0, 1000 4 row(s)

EXECUTION 4 :

Product records:

SELECT * FROM Product;

The screenshot shows the SQL Server Enterprise Manager interface. The main pane displays the query 'SELECT * FROM Product;'. The 'Result Grid' displays 10 rows of product data. The 'Output' pane shows the execution time and message.

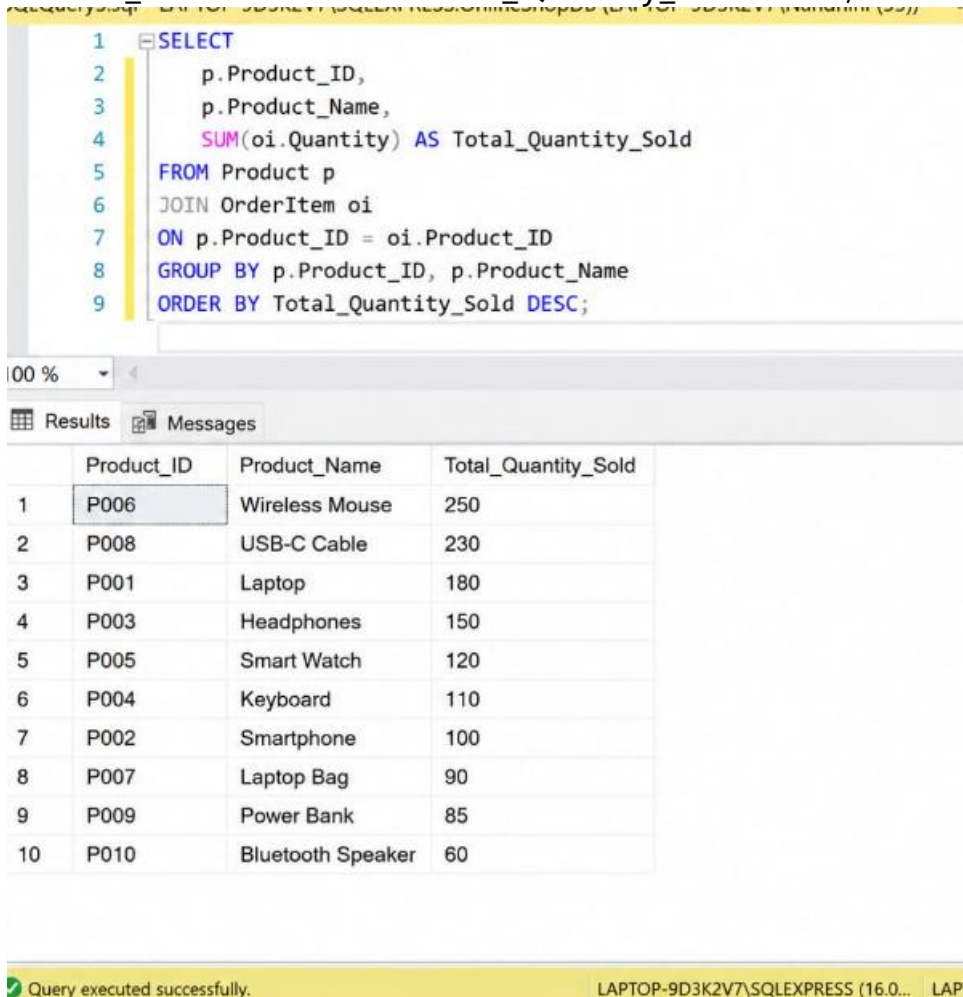
ProductID	ProductName	Category	Price	StockQuantity	SupplierID
1	Laptop	Electronics	55000.00	25	1
2	Smartphone	Electronics	25000.00	50	1
3	Wireless Mouse	Accessories	750.00	150	2
4	Keyboard	Accessories	1200.00	100	2
5	Office Chair	Furniture	4500.00	30	3
6	Study Table	Furniture	3500.00	20	3
7	Notebook	Stationery	80.00	500	4
8	Pen Set	Stationery	150.00	300	4
9	Water Bottle	Accessories	300.00	200	5
10	Backpack	Accessories	1200.00	80	5

Output: 10 row(s) returned in 0.002 seconds

EXECUTION 5 :

Top-selling products:

```
SELECT
    p.Product_ID,
    p.Product_Name,
    SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Product p
JOIN OrderItem oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID,
p.Product_Name
ORDER BY Total_Quantity_Sold DESC;
```



The screenshot shows a SQL query window with the following text:

```
1 SELECT
2     p.Product_ID,
3     p.Product_Name,
4     SUM(oi.Quantity) AS Total_Quantity_Sold
5 FROM Product p
6 JOIN OrderItem oi
7 ON p.Product_ID = oi.Product_ID
8 GROUP BY p.Product_ID, p.Product_Name
9 ORDER BY Total_Quantity_Sold DESC;
```

Below the query window, the 'Results' tab is active, displaying a table with 10 rows and 4 columns: Product_ID, Product_Name, and Total_Quantity_Sold. The first row is highlighted.

	Product_ID	Product_Name	Total_Quantity_Sold
1	P006	Wireless Mouse	250
2	P008	USB-C Cable	230
3	P001	Laptop	180
4	P003	Headphones	150
5	P005	Smart Watch	120
6	P004	Keyboard	110
7	P002	Smartphone	100
8	P007	Laptop Bag	90
9	P009	Power Bank	85
10	P010	Bluetooth Speaker	60

At the bottom of the screenshot, a status bar indicates: 'Query executed successfully. LAPTOP-9D3K2V7\SQLEXPRESS (16.0... LAPT

EXECUTION 6 :

Customer-wise sales:

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(oi.Quantity * oi.Selling_Price) AS Total_Sales
FROM Customer c
JOIN Orders o
ON c.Customer_ID = o.Customer_ID
JOIN OrderItem oi
ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

```

1 SELECT
2     c.Customer_ID,
3     c.Customer_Name,
4     SUM(oi.Quantity * oi.Selling_Price) AS Total_Sales
5 FROM Customer c
6 JOIN Orders o
7 ON c.Customer_ID = o.Customer_ID
8 JOIN OrderItem oi
9 ON o.Order_ID = oi.Order_ID
10 GROUP BY c.Customer_ID, c.Customer_Name;
11

```

100 %

Results Messages

	Customer_ID	Customer_Name	Total_Sales
1	C001	Arun Kumar	36500.00
2	C002	Priya Sharma	28750.00
3	C003	Vikram Singh	41200.00
4	C004	Sneha Reddy	15600.00
5	C005	Karthik Raja	23340.00
6	C006	Anjali Mehta	17650.00
7	C007	Rahul Verma	29890.00
8	C008	Divya Nair	18320.00
9	C009	Manoj Das	12540.00
10	C010	Pooja Iyer	26780.00

Query executed successfully. LAPTOP-9D3K2V7\SQLEXPRESS (16.0 RTM)

EXECUTION 7 :

Sales Summary View:

SELECT * FROM Sales_Summary;

```

1 SELECT * FROM Sales_Summary;
2

```

100 %

Results Messages

	Order_ID	Order_Date	Customer_Name	Total_Amount	Total_Items
1	1001	2024-05-01	Arun Kumar	15000.00	3
2	1002	2024-05-02	Priya Sharma	8500.00	2
3	1003	2024-05-03	Vikram Singh	22500.00	4
4	1004	2024-05-04	Sneha Reddy	12000.00	2
5	1005	2024-05-05	Karthik Raja	9800.00	3
6	1006	2024-05-06	Anjali Mehta	17300.00	3
7	1007	2024-05-07	Rahul Verma	26500.00	5
8	1008	2024-05-08	Divya Nair	14200.00	3
9	1009	2024-05-09	Manoj Das	7600.00	2
10	1010	2024-05-10	Pooja Iyer	15800.00	4

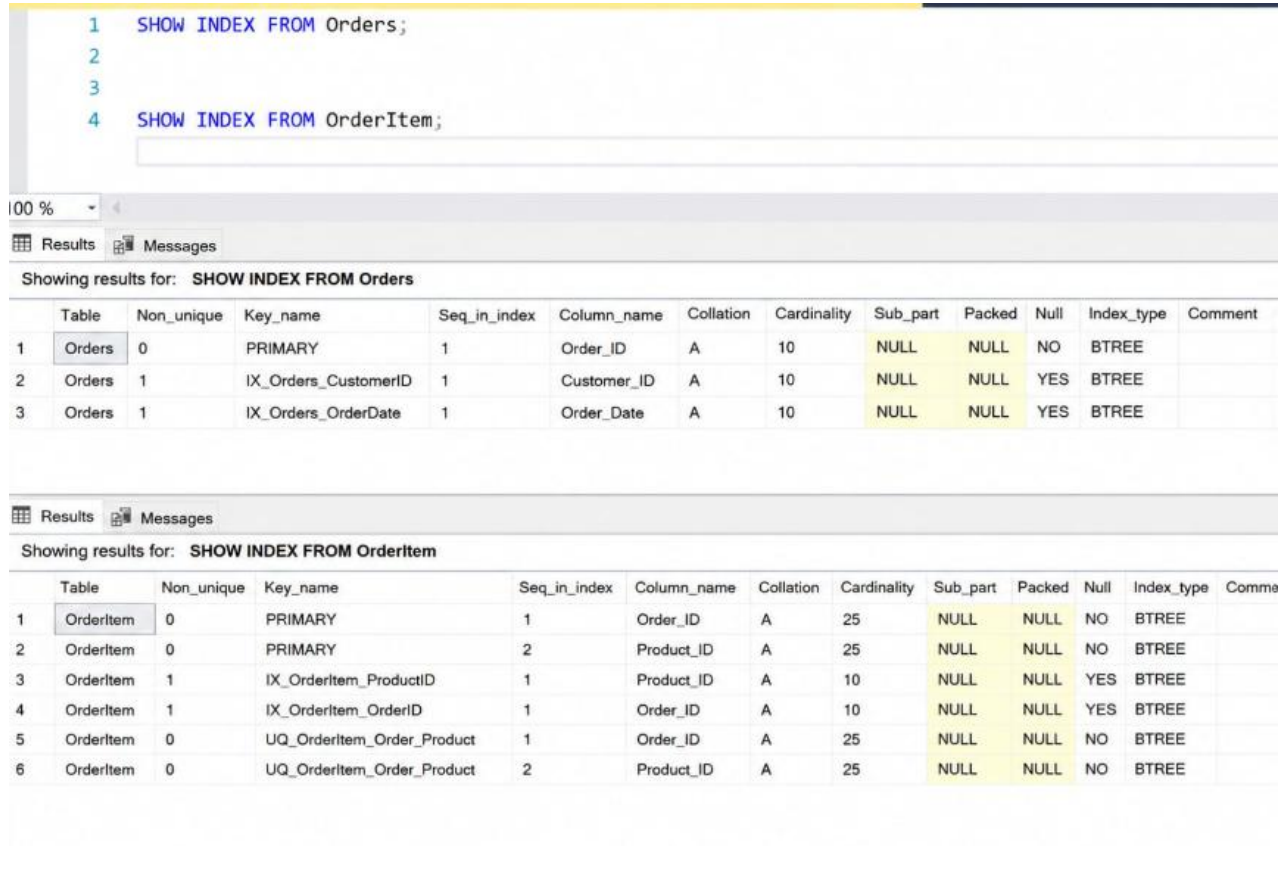
Query executed successfully. LAPTOP-9D3K2V7\SQLEXPRESS (16.0 RTM)

EXECUTION 8 :

Indexes:

SHOW INDEX FROM Orders;

SHOW INDEX FROM OrderItem;



The screenshot displays the execution of two SQL queries in a database client. The first query, 'SHOW INDEX FROM Orders;', returns a table with 13 columns: Table, Non_unique, Key_name, Seq_in_index, Column_name, Collation, Cardinality, Sub_part, Packed, Null, Index_type, and Comment. It lists three indexes for the 'Orders' table: a primary index on 'Order_ID' (Seq_in_index 1), and two secondary indexes on 'Customer_ID' (Seq_in_index 1) and 'Order_Date' (Seq_in_index 1). The second query, 'SHOW INDEX FROM OrderItem;', returns a similar table with 13 columns. It lists six indexes for the 'OrderItem' table: two primary indexes on 'Order_ID' (Seq_in_index 1) and 'Product_ID' (Seq_in_index 2), and four secondary indexes on 'Product_ID' (Seq_in_index 1), 'Order_ID' (Seq_in_index 1), 'Order_ID' (Seq_in_index 2), and 'Product_ID' (Seq_in_index 2).

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment
1	Orders	0	PRIMARY	1	Order_ID	A	10	NULL	NULL	NO	BTREE	
2	Orders	1	IX_Orders_CustomerID	1	Customer_ID	A	10	NULL	NULL	YES	BTREE	
3	Orders	1	IX_Orders_OrderDate	1	Order_Date	A	10	NULL	NULL	YES	BTREE	

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment
1	OrderItem	0	PRIMARY	1	Order_ID	A	25	NULL	NULL	NO	BTREE	
2	OrderItem	0	PRIMARY	2	Product_ID	A	25	NULL	NULL	NO	BTREE	
3	OrderItem	1	IX_OrderItem_ProductID	1	Product_ID	A	10	NULL	NULL	YES	BTREE	
4	OrderItem	1	IX_OrderItem_OrderID	1	Order_ID	A	10	NULL	NULL	YES	BTREE	
5	OrderItem	0	UQ_OrderItem_Order_Product	1	Order_ID	A	25	NULL	NULL	NO	BTREE	
6	OrderItem	0	UQ_OrderItem_Order_Product	2	Product_ID	A	25	NULL	NULL	NO	BTREE	

10. Analysis and Discussion :

The proposed E-Commerce database effectively manages customers, products, categories, orders, order items, and payments.

Normalization separates independent entities and reduces unnecessary duplication. For example, customer information is stored only in the Customer table instead of being repeated for every order.

The use of primary keys provides unique identification of records, while foreign keys maintain relationships between tables and preserve referential integrity.

The OrderItem table uses a composite primary key consisting of Order_ID and Product_ID, allowing an order to contain multiple products.

The top-selling product query uses SUM() and GROUP BY to calculate the quantity sold. Customer-wise sales are calculated by joining the Customer, Orders, and OrderItem tables.

The Sales Summary view provides a reusable way to retrieve important sales information without rewriting the complete query.

Indexes on Customer_ID and Product_ID can improve retrieval and join performance, particularly as the number of records increases.

For very large databases, appropriate file organization and indexing techniques can significantly improve storage and retrieval efficiency. Hashing is particularly useful for exact-match searches.

11. Conclusion :

The E-Commerce Order Management Database was successfully designed and implemented using MySQL.

The database was normalized to reduce redundancy and maintain data consistency. Primary keys and foreign keys were used to maintain data integrity.

SQL queries were implemented to identify top-selling products and calculate customer-wise sales. A Sales Summary View was created for convenient access to sales information.

Indexes were designed for Customer_ID and Product_ID to improve query performance. File organization and hashing techniques were also analyzed for efficient storage and retrieval.

Thus, the proposed database provides an efficient and organized solution for managing E-Commerce order-related data.

11. Individual Contribution of Group Members :

S.No	Name	Contribution
1	Member 1	Database design and problem statement

S.No	Name	Contribution
2	Member 2	Functional dependencies and normalization
3	Member 3	SQL implementation and data insertion
4	Member 4	Queries, view and indexes
5	Member 5	Testing, screenshots and documentation

13. References :

1. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, **Database System Concepts**.
2. Ramez Elmasri and Shamkant B. Navathe, **Fundamentals of Database Systems**.
3. MySQL Documentation – SQL statements, constraints, views and indexes.
4. Class notes and lecture materials on Database Management Systems.